

PRÁCTICA 5

Espectrograma y Filtros Digitales sobre Señal IMU

STFT · FIR · IIR Butterworth · Pipeline PDR comparativo

Plataforma	Python 3.10+ (Windows)	Datos	3 CSV propios capturados en P4 (andando / corriendo / mixto)
Duración	~2,5 horas	Temas	Tema 8 (STFT / Espectrograma) + Tema 9 (FIR / IIR)
Bloques	A (contexto) + B + C + D	Prerreq.	Práctica 4 completada · 3 CSV propios del alumno disponibles

1. Introducción y Contexto

En la Práctica 4 construiste un pipeline PDR (Pedestrian Dead Reckoning) completo: cargabas los datos IMU crudos, eliminabas duplicados, detectabas pasos mediante picos en $|a|$ y reconstruías la trayectoria 2D. El detector funcionaba sobre datos sin ningún tratamiento previo.

En esta práctica el objetivo es doble y usa los 3 CSV que ya grabaste en la sesión anterior con el datalogIMU:

- Analizar la señal en el dominio tiempo-frecuencia mediante el espectrograma (STFT): ver cuándo ocurre cada actividad —andando, corriendo, mixta— y qué la diferencia espectralmente.
- Aplicar filtros digitales FIR e IIR al pipeline de la P4 y medir cuánto mejora el contador de pasos en cada uno de los 3 escenarios grabados.

¿Por qué los 3 CSV son el núcleo de esta práctica?

Cada actividad tiene una huella espectral distinta. Andando: frecuencia fundamental 1,5–2 Hz. Corriendo: 2,5–3 Hz.

El ruido de alta frecuencia (>8 Hz) tiene diferente amplitud en cada caso: al correr, las vibraciones mecánicas son más intensas.

Eso implica que el beneficio del filtrado NO es igual en los 3 archivos. Cuantificar esa diferencia es la pregunta central de la práctica.

Al comparar los 3 resultados aprendes que el diseño de un filtro no es universal: depende de la naturaleza de la señal.

2. Objetivos de Aprendizaje

Al finalizar esta práctica el alumno habrá adquirido las siguientes competencias:

- Generar e interpretar espectrogramas STFT de señal inercial para los 3 escenarios de actividad, identificando frecuencias fundamentales y diferencias visuales entre actividades.
- Aplicar un filtro FIR (Media Móvil, `np.convolve`) y un filtro IIR (Butterworth paso-banda, `scipy.signal.butter + filtfilt`) a la señal `|a|` y comparar su efecto visual y cuantitativo.
- Integrar el filtrado en el pipeline PDR de la P4 y medir la mejora en el conteo de pasos en los 3 CSV, produciendo una tabla comparativa cruda vs. filtrada.
- Justificar la elección del tipo de filtro según la naturaleza de la señal y los recursos del sistema embebido, conectando con los conceptos de FIR/IIR del Tema 9.

3. Material y Ficheros de Partida

3.1 Hardware y software

- PC con Python 3.10+ (Windows) y PyCharm Professional (licencia UVigo)
- Librerías:
 - `pip install numpy pandas matplotlib scipy`

3.2 Ficheros necesarios

Debes tener en la misma carpeta de trabajo los 3 CSV que grabaste en la práctica anterior y renombraste. En esta práctica los llamaremos con los nombres estándar siguientes — si los tuyos tienen otro nombre, ajusta la variable al inicio de cada script:

Variable en el código	Nombre estándar	Descripción
FILE_AND	andando.csv	Recorrido completo andando a paso normal
FILE_COR	corriendo.csv	Recorrido completo corriendo o paso rápido
FILE_MIX	andando_corriendo.csv	Recorrido mixto: mezcla de ambas actividades

⚠ Comprueba tus CSV antes de empezar

Abre cada fichero con un editor de texto y verifica que la primera línea es exactamente:

```
ts_ms,ax,ay,az,gx,gy,gz
```

Si hay líneas en blanco al inicio o al final, elimínalas. Si el separador es punto y coma (;), cámbialas a coma con Buscar y Reemplazar.

Verifica también que tienes al menos 3000 filas en cada fichero (unos 20–30 segundos de datos).

A El datalogIMU.py — Cómo se capturaron los datos

A.1 Descripción del sistema de captura

Los 3 CSV que vas a analizar en esta práctica fueron grabados en la sesión anterior con el programa `datalogIMU.py`, que corre directamente en el M5StickC Plus 2 sobre MicroPython/UIFlow 2.0. Entender cómo funciona la captura es esencial para interpretar correctamente los datos.

El programa implementa una máquina de estados con dos modos:

- Modo GRABACIÓN (BtnA): escribe cada muestra IMU en la flash interna (/flash/imu_log.csv) en tiempo real. Hace flush cada 100 muestras para no perder datos ante un corte de corriente.
- Modo DESCARGA (BtnB): levanta un servidor HTTP en el puerto 80. Conectándose desde el PC a la IP que muestra la pantalla puedes descargar el CSV o borrarlo sin cable USB.

A.2 Formato del CSV generado

Cada fila del CSV corresponde a una lectura del chip MPU-6886:

Columna	Unidad	Descripción
ts_ms	ms	Timestamp del reloj interno del ESP32 (ticks_ms, monótonico)
ax ay az	g	Aceleración en los 3 ejes. En reposo horizontal: az $\approx$ 1,0 g
gx gy gz	°/s	Velocidad angular en los 3 ejes. gz = giro en el plano horizontal (yaw)

A.3 Consideraciones importantes para el análisis

- La tasa de muestreo real NO es constante: el ESP32 en UIFlow 2.0 muestrea a $\sim 100\text{--}130$ Hz pero con cierta variabilidad. Calcula siempre fs real a partir del fichero: $fs = 1000 / dt_medio$.
- Hay muestras duplicadas: el MPU-6886 actualiza su registro interno más lento que la velocidad de lectura del ESP32. Se eliminan en el preprocesado (igual que en la P4).
- El timestamp ts_ms NO empieza en 0: es el valor absoluto de ticks_ms en el momento de la muestra. Convierte a tiempo relativo restando el primer valor.

Fragmento clave del datalogIMU.py — el bucle de grabación

```
if recording and log_file:
    ts    = time.ticks_ms()
    acc   = Imu.getAccel()
    gyro  = Imu.getGyro()
    log_file.write('{}{:.4f},{:.4f},{:.4f},{:.4f},{:.4f},{:.4f}\n'.format(
        ts, acc[0], acc[1], acc[2], gyro[0], gyro[1], gyro[2]))
    if sample_count % 100 == 0:
        log_file.flush()    # Evita pérdida de datos ante corte
```

B Espectrograma STFT — Viendo las 3 actividades

B.1 Objetivo

Generar el espectrograma de la señal $|a|$ para los 3 CSV y comparar visualmente las frecuencias de actividad de cada uno. Relacionar las diferencias con el Principio de Incertidumbre de Heisenberg-Gabor del Tema 8.

B.2 Marco teórico rápido

La **STFT (Short-Time Fourier Transform)** divide la señal en ventanas cortas solapadas, calcula la FFT de cada una y apila los resultados como columnas de una imagen: el **espectrograma**. El eje X es tiempo, el eje Y es frecuencia y el color es la energía en dB.

La resolución frecuencial es $\Delta f = f_s / n_{\text{perseg}}$. Con $n_{\text{perseg}}=256$ y $f_s \approx 120$ Hz obtenemos $\Delta f \approx 0,47$ Hz, suficiente para distinguir andar ($\sim 1,8$ Hz) de correr ($\sim 2,8$ Hz). Aumentar n_{perseg} mejora Δf pero empeora la resolución temporal: no puedes tener ambas a la vez (Heisenberg-Gabor).

B.3 Código de partida — bloque_b_espectrograma.py

```
# bloque_b_espectrograma.py | APS-GRIA Práctica 5
# Genera espectrogramas comparativos para los 3 CSV de actividad

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import signal

# — CONFIGURA AQUÍ LOS NOMBRES DE TUS FICHEROS —
FILE_AND = 'andando.csv'
FILE_COR = 'corriendo.csv'
FILE_MIX = 'andando_corriendo.csv'
FICHEROS = {'Andando': FILE_AND, 'Corriendo': FILE_COR, 'Mixto': FILE_MIX}

# — Parámetros STFT (punto de partida) —
NPERSEG = 256 # TODO: experimenta con 64, 128, 512 en el Exp. B2
NOVERLAP = 128 # 50% overlap

def cargar_y_limpiar(fichero):
    df = pd.read_csv(fichero)
    cols = ['ax', 'ay', 'az', 'gx', 'gy', 'gz']
    df = df.loc[(df[cols] !=
df[cols].shift()).any(axis=1)].reset_index(drop=True)
    df['t_s'] = (df['ts_ms'] - df['ts_ms'].iloc[0]) / 1000.0
    dt = np.diff(df['t_s'].values)
    fs = 1.0 / np.mean(dt)
    df['mag'] = np.sqrt(df['ax']**2 + df['ay']**2 + df['az']**2)
    return df, fs

# — Figura con una fila por CSV (señal + espectrograma) —
fig, axes = plt.subplots(len(FICHEROS), 2, figsize=(16, 4*len(FICHEROS)))

for row, (nombre, fichero) in enumerate(FICHEROS.items()):
    df, fs = cargar_y_limpiar(fichero)
    mag = df['mag'].values
    t = df['t_s'].values
    print(f'{nombre}: fs={fs:.1f} Hz | muestras={len(df)}')

    # Señal temporal
    ax0 = axes[row, 0]
    ax0.plot(t, mag, lw=0.5, color='steelblue')
    ax0.axhline(1.0, color='red', lw=0.8, ls='--', label='1 g')
    ax0.set_title(f'{nombre} — señal |a|')
    ax0.set_ylabel('|a| [g]')
    ax0.set_xlabel('Tiempo [s]')
    ax0.legend(fontsize=8)

    # Espectrograma
    f_bins, t_spec, Sxx = signal.spectrogram(
        mag, fs=fs, window='hann',
        nperseg=NPERSEG, noverlap=NOVERLAP
    )
    Sxx_dB = 10 * np.log10(Sxx + 1e-10)
    ax1 = axes[row, 1]
```

```

im = ax1.pcolormesh(t_spec, f_bins, Sxx_dB,
                    shading='gouraud', cmap='inferno')
ax1.set_ylim([0, min(10, fs/2)])
ax1.set_title(f'{nombre} - espectrograma')
ax1.set_ylabel('Frecuencia [Hz]')
ax1.set_xlabel('Tiempo [s]')
fig.colorbar(im, ax=ax1, label='dB')

# TODO: Añade una línea horizontal en la frecuencia fundamental
#       que identifiques para cada actividad en el Exp. B1.

plt.suptitle('Comparativa espectrogramas - 3 actividades', fontsize=13,
             fontweight='bold')
plt.tight_layout()
plt.savefig('espectrogramas_3csv.png', dpi=150)
plt.show()

```

B.4 Experimentos

Exp. B1 — Frecuencias fundamentales por actividad

Con los parámetros por defecto (NPERSEG=256, NOVERLAP=128), observa los 3 espectrogramas y rellena la tabla:

Fichero	f fundamental [Hz]	¿Armónico 2f visible?	f máx. energía [Hz]	Observación
andando.csv				
corriendo.csv				
andando_corriendo.csv				

Exp. B2 — Dilema de Heisenberg-Gabor

Cambia el valor de NPERSEG y anota el efecto. Usa siempre $\text{NOVERLAP} = \text{NPERSEG} // 2$.

NPERSEG	Δf aprox. [Hz]	Duración ventana [s]	Resolución temporal	Resolución frecuencial
64			Buena/Regular/Mala	Buena/Regular/Mala
128			Buena/Regular/Mala	Buena/Regular/Mala
256			(valor por defecto)	(valor por defecto)
512			Buena/Regular/Mala	Buena/Regular/Mala

Pregunta B2: ¿Con qué NPERSEG se distingue mejor la diferencia de frecuencia entre andar y correr en el fichero mixto? ¿Por qué no se puede mejorar a la vez la resolución temporal y la frecuencial? Responde en un comentario del código.

C Filtros FIR e IIR — Limpiando la señal |a|

C.1 Objetivo

Aplicar un filtro FIR (Media Móvil) y un filtro IIR (Butterworth paso-banda) a la señal $|a|$ de cada CSV. Visualizar el efecto de cada filtro y determinar qué parámetros funcionan mejor para aislar la cadencia de marcha.

C.2 Marco teórico rápido

Filtro FIR — Media Móvil (L muestras): $y[n] = (1/L) \cdot \sum x[n-k], k=0..L-1$ — Actúa como paso-bajo. Fase lineal, siempre estable. Frecuencia de corte aprox.: $f_c \approx f_s / (2 \cdot L)$.

Filtro IIR — Butterworth paso-banda (scipy): `butter(N, [f_low, f_high], btype='bandpass', fs=fs) + filtfilt()` — Corte abrupto, fase cero en offline. Ideal para aislar la banda 0,5–5 Hz donde están los pasos.

¿Por qué filtfilt y no lfilter? `filtfilt` aplica el filtro dos veces (ida y vuelta), consiguiendo fase cero: los picos de la señal filtrada se alinean exactamente con los de la cruda. Si usaras `lfilter`, los picos se desplazarían en el tiempo y el detector de pasos contaría mal.

C.3 Código de partida — bloque_c_filtros.py

```
# bloque_c_filtros.py | APS-GRIA Práctica 5
# Compara FIR (Media Móvil) vs IIR (Butterworth BP) sobre los 3 CSV

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt, spectrogram

# — CONFIGURA AQUÍ —
FILE_AND = 'andando.csv'
FILE_COR = 'corriendo.csv'
FILE_MIX = 'andando_corriendo.csv'
FICHEROS = {'Andando': FILE_AND, 'Corriendo': FILE_COR, 'Mixto': FILE_MIX}

# Parámetros FIR
L = 11 # TODO: prueba L=5, 11, 21 en Exp. C1

# Parámetros IIR
F_LOW = 0.5 # Hz — TODO: experimenta en Exp. C2
F_HIGH = 5.0 # Hz
ORDEN = 4

def cargar_y_limpiar(fichero):
    df = pd.read_csv(fichero)
    cols = ['ax', 'ay', 'az', 'gx', 'gy', 'gz']
    df = df.loc[df[cols] != ]
    df[cols].shift().any(axis=1).reset_index(drop=True)
    df['ts_s'] = (df['ts_ms'] - df['ts_ms'].iloc[0]) / 1000.0
    dt = np.diff(df['ts_s'].values)
    fs = 1.0 / np.mean(dt)
    df['mag'] = np.sqrt(df['ax']**2 + df['ay']**2 + df['az']**2)
    return df, fs

# Procesamos el primer fichero para la figura comparativa de 4 subplots
# (repite el proceso para los otros dos en los experimentos)
nombre_demo = 'Andando'
fichero_demo = FILE_AND
```

```

df, fs = cargar_y_limpiar(fichero_demo)
mag = df['mag'].values
t = df['t_s'].values

# — Filtro FIR: Media Móvil —————
kernel = np.ones(L) / L
mag_fir = np.convolve(mag, kernel, mode='same')

# — Filtro IIR: Butterworth paso-banda —————
b, a = butter(ORDEN, [F_LOW, F_HIGH], btype='bandpass', fs=fs)
mag_iir = filtfilt(b, a, mag) # fase cero

# — Figura de 4 subplots —————
fig, axes = plt.subplots(4, 1, figsize=(14, 12), sharex=True)

axes[0].plot(t, mag, lw=0.5, color='steelblue', label='Cruda')
axes[0].set_ylabel('[g]') ; axes[0].legend(fontsize=8)
axes[0].set_title(f'{nombre_demo} — Comparativa filtros')

axes[1].plot(t, mag_fir, lw=0.8, color='darkorange', label=f'FIR MA L={L}')
axes[1].set_ylabel('[g]') ; axes[1].legend(fontsize=8)

axes[2].plot(t, mag_iir, lw=0.8, color='green', label='IIR Butterworth BP')
axes[2].axhline(0, color='gray', lw=0.5, ls='--')
axes[2].set_ylabel('[g]') ; axes[2].legend(fontsize=8)

# Subplot 4: espectrograma de la señal IIR (confirma qué frecuencias quedaron)
f_b, t_s, Sxx = spectrogram(mag_iir, fs=fs, window='hann',
                             nperseg=256, noverlap=128)
axes[3].pcolormesh(t_s, f_b, 10*np.log10(Sxx+1e-10),
                   shading='gouraud', cmap='inferno')
axes[3].set_ylim([0, min(10, fs/2)])
axes[3].set_ylabel('Freq [Hz]') ; axes[3].set_xlabel('Tiempo [s]')
axes[3].set_title('Espectrograma señal IIR')

plt.tight_layout()
plt.savefig(f'filtros_{nombre_demo.lower()}.png', dpi=150)
plt.show()

print(f'fs={fs:.1f} Hz | FIR L={L} | IIR BP [{F_LOW}-{F_HIGH}] Hz orden {ORDEN}')

```

C.4 Experimentos

Exp. C1 — Efecto de L en el filtro FIR (usa andando.csv)

Cambia L y anota el retardo visual de los picos y si se pierden o distorsionan.

L	fc aprox. [Hz]	Retardo [ms]	¿Picos distorsionados?	Valoración
5		$\approx L/(2 \cdot fs) \cdot 1000$		
11				(por defecto)
21				

Exp. C2 — Selección de banda para el IIR (usa corriendo.csv)

Prueba distintas bandas y observa si se conservan los picos de paso. Usa siempre ORDEN=4 y filtfilt.

f_low [Hz]	f_high [Hz]	¿Pasos conservados?	¿Ruido residual?	Observación
0,5	3,0			
0,5	5,0			(por defecto)
1,0	5,0			
0,5	8,0			

Exp. C3 — Repetir figura de 4 subplots para corriendo.csv y andando_corriendo.csv

Con los parámetros que hayas elegido como óptimos en C1 y C2, genera la figura de 4 subplots para los 3 ficheros. Observa y anota en comentarios: ¿en qué fichero el filtro IIR atenúa más el ruido visible en el espectrograma?

D Integración en el pipeline PDR — Comparativa sobre los 3 CSV

D.1 Objetivo

Modificar el detector de pasos de la Práctica 4 para operar sobre la señal $|a|$ filtrada con el IIR Butterworth y comparar el conteo de pasos crudo vs. filtrado en los 3 CSV. La tabla comparativa final es la pieza central de la entrega.

D.2 Cambios en el pipeline respecto a la P4

Pipeline P4 original → Pipeline P5 mejorado

P4: CSV → limpiar duplicados → $|a|$ → find_peaks($|a|$, height=1.2) → pasos → PDR

P5: CSV → limpiar duplicados → $|a|$ → IIR BP filtfilt → find_peaks($|a|_{\text{filt}}$, height=UMBRAL) → pasos → PDR

Cambio importante: la señal IIR paso-banda está centrada en 0 (elimina la componente DC de 1 g).

Por eso el UMBRAL deja de ser ~1,2 g y pasa a ser un valor positivo pequeño (~0,1–0,3 g).

Ajusta el UMBRAL de forma empírica para cada fichero o usa un valor único que funcione en los 3.

D.3 Código de partida — bloque_d_pipeline.py

```
# bloque_d_pipeline.py | APS-GRIA Práctica 5
# Pipeline PDR con filtrado IIR — comparativa sobre los 3 CSV

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt, find_peaks, spectrogram as spec_fn
```



```

# — CONFIGURA AQUÍ —————
FILE_AND = 'andando.csv'
FILE_COR = 'corriendo.csv'
FILE_MIX = 'andando_corriendo.csv'
FICHEROS = {'Andando': FILE_AND, 'Corriendo': FILE_COR, 'Mixto': FILE_MIX}

# Parámetros filtro IIR (usa los mejores de tu Exp. C2)
F_LOW = 0.5
F_HIGH = 5.0
ORDEN = 4

# Parámetros detector de pasos
UMBRAL_CRUDO = 1.20 # g – igual que en la P4
UMBRAL_FILTRADO = 0.15 # g – TODO: ajusta para cada fichero (Exp. D1)
PASO_M = 0.75 # metros – longitud de paso fija

# — Función de procesamiento completo —————
def procesar(fichero, nombre):
    df = pd.read_csv(fichero)
    cols = ['ax', 'ay', 'az', 'gx', 'gy', 'gz']
    df = df.loc[(df[cols] !=
df[cols].shift()).any(axis=1)].reset_index(drop=True)
    df['t_s'] = (df['ts_ms'] - df['ts_ms'].iloc[0]) / 1000.0
    dt = np.diff(df['t_s'].values)
    fs = 1.0 / np.mean(dt)
    df['mag'] = np.sqrt(df['ax']**2 + df['ay']**2 + df['az']**2)
    mag = df['mag'].values
    t = df['t_s'].values

    # Filtro IIR
    b, a = butter(ORDEN, [F_LOW, F_HIGH], btype='bandpass', fs=fs)
    mag_filt = filtfilt(b, a, mag)

    # Detección de pasos
    dist = int(0.4 * fs) # 400 ms mínimo entre pasos
    picos_c, _ = find_peaks(mag, height=UMBRAL_CRUDO, distance=dist)
    picos_f, _ = find_peaks(mag_filt, height=UMBRAL_FILTRADO, distance=dist)

    dist_c = len(picos_c) * PASO_M
    dist_f = len(picos_f) * PASO_M

    print(f'--- {nombre} ---')
    print(f' fs = {fs:.1f} Hz | muestras = {len(df)}')
    print(f' Pasos CRUDO (umbral={UMBRAL_CRUDO}): {len(picos_c):4d} →
{dist_c:.1f} m')
    print(f' Pasos FILTRADO (umbral={UMBRAL_FILTRADO}): {len(picos_f):4d} →
{dist_f:.1f} m')
    print()

    # Figura comparativa para este fichero
    fig, axes = plt.subplots(3, 1, figsize=(14, 9), sharex=True)
    axes[0].plot(t, mag, lw=0.5, color='steelblue', label='Cruda')
    axes[0].plot(t[picos_c], mag[picos_c], 'rv', ms=5,
label=f'Pasos crudos ({len(picos_c)})')
    axes[0].set_ylabel('g') ; axes[0].legend(fontsize=8)
    axes[0].set_title(f'{nombre} – Cruda (baseline P4)')

    axes[1].plot(t, mag_filt, lw=0.8, color='green', label='IIR BP filtrada')
    axes[1].plot(t[picos_f], mag_filt[picos_f], 'r^', ms=5,
label=f'Pasos filtrados ({len(picos_f)})')
    axes[1].axhline(UMBRAL_FILTRADO, color='red', lw=0.8, ls='--',
label=f'Umbral={UMBRAL_FILTRADO} g')

```

```

axes[1].set_ylabel('[g]') ; axes[1].legend(fontsize=8)
axes[1].set_title(f'{nombre} – Filtrada IIR (P5)')

f_b, t_s, Sxx = spec_fn(mag_filt, fs=fs, window='hann',
                        nperseg=256, noverlap=128)
axes[2].pcolormesh(t_s, f_b, 10*np.log10(Sxx+1e-10),
                  shading='gouraud', cmap='inferno')
axes[2].set_ylim([0, min(10, fs/2)])
axes[2].set_ylabel('Freq [Hz]') ; axes[2].set_xlabel('Tiempo [s]')

plt.tight_layout()
plt.savefig(f'pipeline_{nombre.lower()}.png', dpi=150)
plt.show()

return len(picos_c), len(picos_f), dist_c, dist_f

# — Ejecutar sobre los 3 ficheros —————
resultados = {}
for nombre, fichero in FICHEROS.items():
    resultados[nombre] = procesar(fichero, nombre)

# — Resumen final en consola —————
print('=' * 60)
print(f'{"Fichero":<12} {"Crudo":>8} {"Filtrado":>10} {"Mejora":>10}')
print('-' * 60)
for nombre, (pc, pf, dc, df_) in resultados.items():
    mejora = (pf - pc) / pc * 100 if pc > 0 else 0
    print(f'{nombre:<12} {pc:>8} {pf:>10} {mejora:>+9.1f}%')
print('=' * 60)

```

D.4 Experimentos

Exp. D1 — Ajuste del umbral sobre señal filtrada

La señal IIR paso-banda está centrada en 0. Prueba los siguientes umbrales en andando.csv y anota el conteo:

UMBRAL [g]	Pasos andando	Pasos corriendo	Pasos mixto	¿Falsos positivos?
0,10				
0,15				(por defecto)
0,20				
0,30				

Exp. D2 — Tabla comparativa final (pieza central de la entrega)

Con el umbral que consideras óptimo, rellena esta tabla completa. Es el resultado que el evaluador buscará en tu código:

Fichero	Pasos crudos	Pasos filtrados	Δ pasos	Dist. filtrada [m]	¿Mejora notable?
andando.csv					
corriendo.csv					
andando_corriendo.csv					

Exp. D3 — Pregunta de análisis comparativo

Responde en un comentario en el código (al menos 4–5 líneas con tu razonamiento):

- ¿En qué de los 3 ficheros produce el filtrado IIR la mayor diferencia en el conteo de pasos? ¿Por qué crees que es así? ¿Tiene que ver con la amplitud del ruido de alta frecuencia que viste en los espectrogramas del Bloque B?
- ¿El cambio en el número de pasos indica que el filtrado elimina falsos positivos (pasos fantasma) o que también puede eliminar pasos reales? ¿Cómo lo sabes?

4. Entrega en Moovi

4.1 Qué debes subir

La entrega consiste en un único fichero Python con el código del Bloque D completo. Incluye en ese fichero las figuras de todos los bloques (las genera y guarda como PNG al ejecutarse) y todos los comentarios de los experimentos.

✓ Sí debes subir	✗ NO debes subir
bloque_d_pipeline.py con los 3 CSV procesados	Los ficheros CSV (son datos, no código)
Todos los experimentos D1, D2, D3 comentados en el código	Los ficheros PNG por separado
Los parámetros justificados (UMBRAL, F_LOW, F_HIGH) con comentarios	Los scripts de bloques B y C por separado
Respuesta a al menos una pregunta de reflexión del apéndice	Ficheros ZIP, carpetas o notebooks .ipynb

4.2 Nombre del fichero

```
apellido1_apellido2_nombre_p5_filtros_imu.py
```

Sin tildes, espacios ni mayúsculas. Ejemplo: garcia_lopez_juan_p5_filtros_imu.py

4.3 Rúbrica de evaluación

Criterio	Qué se valora	Puntos
C1. Espectrograma × 3	Se generan 3 espectrogramas (andando, corriendo, andando+corriendo) sobre la señal a . Se identifican y anotan correctamente las frecuencias fundamentales en cada uno.	2,5
C2. Comparativa filtros	Figura de 4 subplots (cruda, FIR MA, IIR Butterworth BP, espectrograma filtrado) sobre al menos uno de los 3 CSV. Los parámetros L y banda de paso están justificados.	2,5

Criterio	Qué se valora	Puntos
C3. Pipeline PDR mejorado	El detector de pasos de la P4 se ejecuta sobre la señal IIR filtrada en los 3 CSV. Se reporta el conteo crudo vs. filtrado en los 3 casos con la tabla comparativa.	2,5
C4. Análisis comparativo	Se responde de forma razonada a la pregunta comparativa: ¿En qué fichero mejora más el filtrado? ¿Por qué? El código incluye comentarios propios que no son el texto base.	1,5
C5. Reflexión	Se responde al menos a una pregunta de reflexión del Bloque D en forma de comentario en el código.	1,0
TOTAL — Nota mínima para aprobar: 5,0		10,0

4.4 Plazo y condiciones

- Plazo: 7 días naturales desde la sesión de práctica (cierre automático a las 23:59).
- Entrega individual. El código se comparará automáticamente para detectar similitud > 80 %.
- El fichero debe ejecutarse sin errores con los 3 CSV en la misma carpeta (Windows, Python 3.10+).
- El UMBRAL_FILTRADO debe ser distinto a 0.15 y estar justificado con un comentario.

5. Preguntas de Reflexión

Incluye la respuesta a al menos una de estas preguntas como comentario en tu fichero de entrega. El comentario debe mostrar tu razonamiento propio.

#	Pregunta
R1	El filtro FIR Media Móvil con $L=11$ introduce un retardo de grupo de $L/2 = 5$ muestras. Con $f_s \approx 120$ Hz, ¿cuántos milisegundos son? ¿Sería un problema en un sistema de detección en tiempo real en el ESP32?
R2	¿Por qué se usa <code>filtfilt</code> en lugar de <code>lfilter</code> para el Butterworth? ¿Qué consecuencia concreta tiene el desfase que introduce <code>lfilter</code> sobre el detector de picos <code>find_peaks</code> ?
R3	En el fichero <code>corriendo.csv</code> , el ruido de alta frecuencia (>8 Hz) es más intenso que en <code>andando.csv</code> . ¿Por qué? ¿Tiene que ver con el impacto del balón, la frecuencia de muestreo o ambas cosas?
R4	Si quisieras portar el filtro IIR Butterworth al ESP32 para detectar pasos en tiempo real en MicroPython, ¿por qué NO podrías usar <code>filtfilt</code> ? ¿Qué usarías en su lugar y con qué coste computacional?
R5	El espectrograma del fichero mixto (<code>andando+corriendo</code>) muestra una transición entre dos bandas de frecuencia. ¿Podrías diseñar un clasificador muy simple (sin IA) basado solo en la energía espectral en la banda 1–2 Hz vs. 2,5–4 Hz para distinguir automáticamente los dos tramos? Descríbelo.

Apéndice — Referencia Rápida de Funciones

Función	Módulo	Uso en la práctica	Ejemplo clave
spectrogram()	scipy.signal	STFT → devuelve (f_bins, t_spec, Sxx)	f,t,S = spectrogram(mag, fs=fs, nperseg=256)
pcolormesh()	matplotlib	Dibuja Sxx como mapa de calor	plt.pcolormesh(t, f, 10*log10(S+1e-10), cmap='inferno')
butter()	scipy.signal	Diseña filtro IIR Butterworth	b,a = butter(4, [0.5,5], 'bandpass', fs=fs)
filtfilt()	scipy.signal	Aplica IIR con fase cero (offline)	y = filtfilt(b, a, x)
np.convolve()	numpy	Aplica filtro FIR por convolución directa	y = np.convolve(x, np.ones(L)/L, 'same')
find_peaks()	scipy.signal	Detecta picos con restricciones	pk,_ = find_peaks(y, height=0.15, distance=int(0.4*fs))
np.mean(dt)	numpy	Calcula fs real del CSV	fs = 1.0 / np.mean(np.diff(t_s))

FIR vs. IIR — ¿Cuándo usar cada uno?

FIR Media Móvil (L=11): simple, sin riesgo de inestabilidad, fase lineal, fácil de portar al ESP32 como buffer circular de L enteros.

IIR Butterworth orden 4: corte mucho más abrupto, mejor rechazo del ruido, pero requiere conocer fs exacta.

→ En análisis offline (Python + filtfilt): usa Butterworth. Corte limpio y fase cero.

→ En tiempo real en ESP32 (MicroPython): usa FIR o EWMA IIR de orden 1 ($\alpha \cdot x + (1-\alpha) \cdot y_{\text{ant}}$).

El Butterworth discreto de orden 4 requiere guardar 4 valores de estado y tiene riesgo de overflow.